Créer un bon fichier `AGENTS.md`

Guide général pour travailler avec Codex

Création : 03/11/2026 · Version 1.0 · Mise à jour : 03/11/2026

Un bon `AGENTS.md` donne à Codex des consignes durables, concrètes et adaptées à votre façon de travailler. Ce guide propose une méthode utilisable pour un logiciel, une analyse de données, un rapport ou un autre projet disposant d'un espace de fichiers.

Comprendre son rôle

Selon OpenAI, `AGENTS.md` sert à transmettre les conventions et instructions réutilisables d'un projet. Il peut préciser son organisation, les commandes utiles, les contraintes et la manière de vérifier un travail. OpenAI recommande un fichier court et exact, enrichi à partir des difficultés réellement rencontrées. [2, 3]

Le `README` présente le projet à ses lecteurs : objectif, contenu et utilisation. `AGENTS.md` indique comment Codex doit y travailler. Le message de la conversation décrit la tâche du moment : par exemple, corriger un calcul ou revoir une section. Évitez de recopier toute la documentation dans les trois endroits (readme, agents et conversation).

Commencer simplement

- Créez un fichier texte nommé exactement `AGENTS.md`, à la racine du projet, c'est-à-dire dans son dossier principal. Markdown est du texte avec quelques signes de mise en forme, comme # pour un titre.
- Choisissez quelques règles que vous voulez retrouver à chaque tâche. Gardez la demande ponctuelle et le calendrier dans leurs documents habituels.
- Ajoutez seulement des chemins, des outils et des commandes qui existent et dont vous avez vérifié l'utilité.

La présence de ce fichier ne prouve ni la justesse d'une production ni le respect de toutes les consignes. L'humain examine les résultats et décide de les accepter. Le fichier ne donne pas à lui seul accès à un outil, à un compte ou à une ressource externe.

1. Choisir les informations utiles

Pour chaque rubrique ci-dessous, écrivez uniquement ce qui aide à prendre une décision ou à accomplir une action. Vous pouvez supprimer une rubrique inutile.

- Finalité et repères : quel résultat cherche-t-on ? Quels fichiers ou sources font référence ?
- Méthode : que lire en premier ? Quelles modifications restent dans le périmètre ? Quand signaler une ambiguïté ?
- Conventions : langue, organisation, formats, vocabulaire et pratiques propres au projet.
- Vérification : comment constater que le résultat correspond à la demande ? Quelles commandes ou vérifications manuelles utiliser ?
- Limites et restitution : quelles actions demandent une décision humaine ? Quelles preuves et limites présenter à la fin ?

Transformer une intention en consigne

Une formulation utile associe une situation, une action et une preuve observable. Les reformulations suivantes sont des exemples proposés dans ce guide.

Formulation trop vague	**Consigne exploitable**
« Fais un travail de qualité. »	« Vérifier le résultat avec le contrôle indiqué, puis rapporter le résultat observé. »
« Lis toute la documentation. »	« Commencer par le README et les sources citées pour la tâche ; signaler une source inaccessible. »
« Ne fais jamais d'erreur. »	« Ne pas inventer une information manquante ; distinguer hypothèse et fait vérifié. »
« Demande toujours avant d'agir. »	« Avancer sur les changements réversibles prévus ; demander avant une publication non autorisée. »

Pour un logiciel, une preuve peut être un test exécuté avec son résultat. Pour une analyse, ce peut être un contrôle des totaux et des valeurs manquantes. Pour un rapport, vérifiez les sources, la cohérence des affirmations et le document rendu. Le contrôle dépend de l'artefact produit.

Si une commande est utile, notez aussi le dossier depuis lequel l'exécuter et les prérequis. N'écrivez pas « lancer `npm test` » simplement parce que cette commande apparaît dans un exemple : elle doit correspondre au projet réel.

2. Un exemple de départ à adapter

Cet exemple est une proposition de rédaction, pas un modèle officiel d'OpenAI. Il n'exige aucun outil particulier. Copiez son contenu dans `AGENTS.md`, puis ajustez les règles à votre projet.

```
# Consignes de travail

## Repères
- Lire le README s'il existe, puis les fichiers utiles à la tâche.
- Identifier les sources de référence avant de modifier le contenu.
- Distinguer les faits établis, les hypothèses et les décisions.

## Manière de travailler
- Limiter les changements à la demande en cours.
- Préserver les modifications existantes de l'utilisateur.
- Avancer par changements faciles à examiner et à annuler.
- Signaler une ambiguïté qui change le résultat attendu.
- Proposer une option motivée si une décision manque.

## Vérification
- Choisir un contrôle pertinent pour chaque changement important.
- Exécuter les contrôles disponibles et examiner leurs résultats.
- Distinguer contrôle réussi, échec observé et contrôle non réalisé.
- Ne pas annoncer comme vérifié un résultat non contrôlé.

## Limites
- Ne pas inclure de secrets ou de données privées dans les livrables.
- Ne pas publier ni supprimer définitivement sans autorisation.
- Signaler un besoin d'outil ou d'accès manquant.

## Restitution
- Résumer ce qui a changé et pourquoi.
- Indiquer les contrôles exécutés et leurs résultats.
- Mentionner les limites non résolues et les décisions à prendre.
```

Pour le rendre plus utile, ajoutez ensuite vos véritables chemins de référence, conventions et moyens de vérification. Supprimez une règle si elle est trop générale, sans effet concret ou déjà mieux définie ailleurs.

3. Installer, essayer et améliorer

Rendre le fichier disponible

1. Créez `AGENTS.md` avec un éditeur de texte dans le dossier principal du projet. Vérifiez que son nom ne se termine pas par `.txt` et que ce n'est pas un document Word renommé.
2. En local, enregistrez-le dans le projet ouvert avec Codex. Dans le Cloud, enregistrez-le dans le dépôt utilisé par l'environnement et vérifiez que cette version y est effectivement présente.
3. Pour vérifier une modification des consignes, démarrez une nouvelle session ou tâche sur la bonne version du projet : OpenAI décrit une découverte des fichiers au démarrage d'une exécution. Ne supposez pas qu'un chat déjà ouvert a automatiquement rechargé le fichier. [1]

Vérifier sur une petite tâche

Vous pouvez commencer par cette demande :

« Sans modifier les fichiers, indique quels fichiers `AGENTS.md` tu as pris en compte. Résume les consignes pertinentes pour ma demande et précise les contrôles que tu utiliserais. Signale les fichiers ou outils qui te manquent. »

Ce résumé aide à repérer une mauvaise version ou une incompréhension ; il ne prouve pas que les prochaines actions respecteront toutes les règles. Donnez ensuite une petite tâche réversible et examinez les changements, les preuves annoncées et les limites.

Entretenir sans allonger inutilement

- Lorsqu'une erreur se répète, demandez-vous quelle instruction précise aurait pu l'éviter. Ajoutez-la seulement si elle sera utile à nouveau.
- Corrigez les chemins et commandes devenus faux ; retirez les règles périmées et les contradictions.
- Gardez le fichier principal court. Renvoyez vers une documentation ciblée pour les procédures longues et précisez quand la consulter.

OpenAI recommande cette amélioration progressive à partir du travail réel. [2, 3] Une règle ajoutée doit rester compréhensible et proportionnée ; demander une confirmation pour chaque action courante peut ralentir le travail sans améliorer le résultat.

4. Éviter les confusions

Plusieurs fichiers : seulement si nécessaire

Codex peut combiner des consignes personnelles, celles du projet et celles de dossiers plus proches de son répertoire de travail. La documentation décrit un parcours depuis la racine jusqu'à ce répertoire : les consignes les plus proches prennent le dessus en cas de conflit. Dans un même dossier, `AGENTS.override.md` est prioritaire sur `AGENTS.md` ; les deux ne sont pas additionnés. [1]

Pour débuter, un seul `AGENTS.md` à la racine suffit généralement. Un fichier supplémentaire se justifie lorsqu'un dossier a des règles propres. Ne répartissez pas les mêmes règles partout et ne supposez pas que tous les sous-dossiers sont parcourus d'un seul coup.

Si les consignes semblent ignorées

- Vérifiez le nom exact du fichier, son enregistrement et le projet réellement ouvert.
- Contrôlez la version du dépôt disponible dans l'environnement utilisé.
- Cherchez une consigne contradictoire ou un `AGENTS.override.md` applicable.
- Essayez une nouvelle tâche et vérifiez une action concrète, pas seulement une déclaration de l'agent.

Un lien écrit dans `AGENTS.md` ne rend pas son contenu accessible. Si une source manque, rendez-la disponible par un moyen autorisé ou fournissez les éléments nécessaires. Les possibilités dépendent de la version de Codex, de l'environnement et des permissions.

Avant de conserver une nouvelle règle

Est-elle durable, précise, vérifiable et compatible avec les autres consignes ? Ses références existent-elles ? L'effort de vérification est-il adapté à l'enjeu ? Une autre personne comprendrait-elle quand et comment l'appliquer ?

Sources officielles

[1] OpenAI — Custom instructions with `AGENTS.md`

[2] OpenAI — Best practices

[3] OpenAI — Customization

Sources consultées le 03/11/2026. Les explications de fonctionnement renvoient à OpenAI ; la méthode, les reformulations et le fichier d'exemple sont une synthèse pédagogique originale. L'humain conserve la décision finale.